Quest: Query-Aware Sparsity for Efficient Long-Context LLM Inference

Jiaming Tang [*1 2] **Yilong Zhao** [*1 3] **Kan Zhu** [3] **Guangxuan Xiao** [2] **Baris Kasikci** [3] **Song Han** [2 4]

Abstract

As the demand for long-context large language models (LLMs) increases, models with context windows of up to 128K or 1M tokens are becoming increasingly prevalent. However, long-context LLM inference is challenging since the inference speed decreases significantly as the sequence length grows. This slowdown is primarily caused by loading a large KV cache during self-attention. Previous works have shown that a small portion of critical tokens will dominate the attention outcomes. However, we observe the criticality of a token highly depends on the query. To this end, we propose Quest, a query-aware KV cache selection algorithm. Quest keeps track of the minimal and maximal Key values in KV cache pages and estimates the criticality of a given page using Query vectors. By only loading the Top-K critical KV cache pages for attention, Quest significantly speeds up self-attention without sacrificing accuracy. We show that Quest can achieve up to 7.03× self-attention speedup, which reduces inference latency by 2.23× while performing well on tasks with long dependencies with negligible accuracy loss. Code is available at https://github.com/mit-han-lab/Quest.

1. Introduction

The rapid evolution of Large Language Models (LLMs) has shaped our daily lives. With the increasing demand for multi-round conversations and long document queries, the maximum context length of LLMs has dramatically grown from 2K to 1M (Liu et al., 2024a; Peng et al., 2023; Tworkowski et al., 2023). The 128k context length GPT-4 model has already been deployed in large-scale serving, which is equivalent to 300 pages of text (OpenAI, 2023).

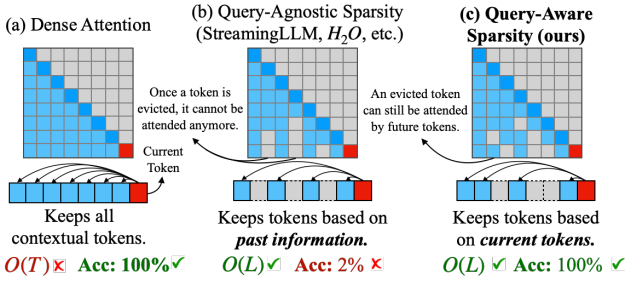

Figure 1. Comparison between Dense Attention(a), Query-Agnostic Sparsity (b) and Quest's Query-aware Sparsity (c). Quest significantly speeds up self-attention while maintaining high accuracy by dynamically determining the critical tokens based on the current query. T represents the total sequence length and L represents the number of critical tokens for attention.

However, processing long-context requests is challenging. Due to the auto-regressive nature of LLMs, generating one token would require reading the entire KV cache. For Llama 7B model (Touvron et al., 2023) with 32k context length, the KV cache can occupy 16GB of space, which requires at least 11 ms to read, which contributes to more than 50% of the inference latency[*], limiting the overall throughput.

Despite the increasingly large size of the KV cache, previous works have shown that a small portion of the tokens can dominate the accuracy of token generation (Zhang et al., 2023b; Ge et al., 2024). Therefore, we can dramatically reduce the inference latency by only loading the critical tokens, while still maintaining accuracy. Thus, it is essential to identify critical portions of the KV cache.

In this work, we further observe that the criticality of the tokens can change with different query tokens. As shown in Fig. 2, the critical tokens vary a lot with different queries. Therefore, we need a dynamic and efficient approach to determine which portion of the KV cache needs to be attended to. To this end, we propose Quest, a query-aware criticality estimation algorithm for long-context LLM inference that efficiently and effectively identifies critical KV cache tokens and performs self-attention selectively on chosen tokens, as shown in Fig. 1.

To reduce the overhead of KV cache criticality estimation,

[*]Equal contribution [1]Shanghai Jiao Tong University [2]MIT [3]University of Washington [4]NVIDIA. Correspondence to: Song Han <songhan@mit.edu>, Baris Kasikci <baris@cs.washington.edu>.

Proceedings of the 41st International Conference on Machine Learning, Vienna, Austria. PMLR 235, 2024.

[*]Tested with FP16 FlashInfer implementation on an RTX4090

Quest manages KV cache at page granularity (Kwon et al., 2023). For each page, Quest utilizes maximum and minimum values of each feature dimension of the Key vector as the metadata to represent token information. During inference, Quest considers both the Query vector and the metadata to estimate each page's criticality. Given all criticality scores of the pages, Quest chooses Top-K pages to perform approximate self-attention, where K is a preset constant (e.g. 128, 256). By reducing the memory movement from the entire KV cache to metadata and constant K pages, Quest significantly accelerates inference.

We evaluate both the accuracy and efficiency of Quest. Since Quest dynamically decides the criticality of the tokens, Quest achieves better accuracy for a given degree of KV cache sparsity than baselines on PG19 dataset (Rae et al., 2019), passkey retrieval task (Peng et al., 2023), and LongBench (Bai et al., 2023) with 256 to 4K token budgets. For 32K context, Quest achieves $7.03\times$ self-attention latency reduction compared to FlashInfer (Ye et al., 2024). Our end-to-end framework demonstrates that Quest can have $2.23\times$ inference speedup compared to FlashInfer (Ye et al., 2024) with 4-bit weight quantization. In summary, we make the following contribution:

- An analysis of the self-attention mechanism that pinpoints the importance of query-aware sparsity.
- Quest, an efficient and accurate KV cache acceleration algorithm, which exploits query-aware sparsity by dedicated operator designs and implementations.
- A comprehensive evaluation of Quest, demonstrating up to $7.03\times$ self-attention latency reduction and $2.23\times$ end-to-end latency improvement.

2. Related Work

2.1. Long-context Model

As the demand for long-context models increases, many works have focused on extending the context window of LLMs. Currently, many models utilize Rotary Position Embeddings (RoPE) (Su et al., 2023), and by different scaling methods of RoPE with fine-tuning, the window size of the original 4k Llama-2 has been expanded to 32k for LongChat (Li et al., 2023) and 128k for Yarn-Llama-2 (Peng et al., 2023). Through length extrapolation, the context windows of models reached beyond 1M (Liu et al., 2024b). Beyond open-source models, GPT-4 Turbo supports lengths of up to 128k, while Claude-2 supports up to 200k (OpenAI, 2024; Anthropic, 2024). With models increasingly capable of handling long input, this poses challenges for inference efficiency. Quest aims to boost long-context inference by exploiting query-aware KV cache sparsity.

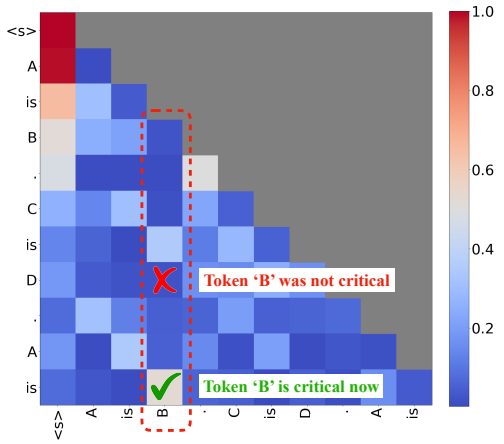

Figure 2. The attention map of prompt "A is B. C is D. A is". Each row represents the attention scores of previous tokens queried by the tokens on the left. When queried with "D", token "B" has a low attention score, showing "B" is not critical for generation. However, the "is" strongly attends to "B". Therefore, the criticality of tokens strongly correlates with the current query token.

2.2. KV Cache Eviction Algorithm

For long-context LLM inference and serving scenarios, the huge size of the KV cache results in significant time and space overheads. Many previous efforts have been dedicated to compressing the size of the KV cache to accelerate attention and reduce memory usage. H2O (Zhang et al., 2023b) retains a limited budget of the important KV cache based on the sum of historical attention scores. FastGen (Ge et al., 2024) further refines the types of tokens, applying a more sophisticated strategy for selecting the KV cache to keep. TOVA (Oren et al., 2024) simplifies the policy by deciding which tokens to permanently discard based solely on the current query. StreamingLLM (Xiao et al., 2023) handles infinitely long texts with attention sinks and a finite KV cache. These methods decide which parts of the KV cache to discard based on historical information or current states, but discarded tokens might be important for future tokens, which may cause the loss of important information. To mitigate this issue, SparQ (Ribar et al., 2023) computes approQximate attention scores by channel pruning and selects important tokens through them. However, this approach has not been widely validated for tasks with long dependencies, and the channel-level sparsity might pose challenges to practical acceleration. Therefore, we propose Quest, which retains all of the KV cache and selects part of the KV cache based on the current query to accelerate long-context self-attention without accuracy degradation.

3. Methodlogy

In this section, we first motivate Quest by analyzing the breakdown of inference cost and self-attention properties.

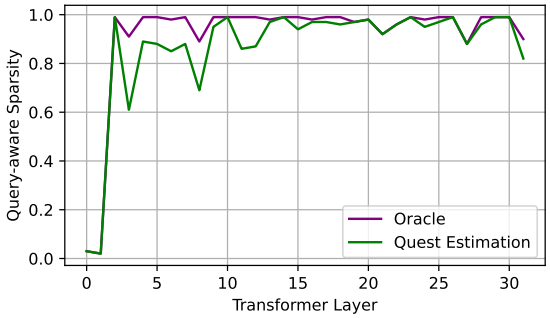

Figure 3. The query aware sparsity for each layer in LongChat-7B model. We measure the sparsity by eliminating KV cache tokens while making sure the perplexity on PG19 increases less than 0.01. For the first two layers, the sparsity is below 10%, while for the rest of the layers, the sparsity is larger than 90%, showing great potential for optimization. Quest closely aligns with the oracle.

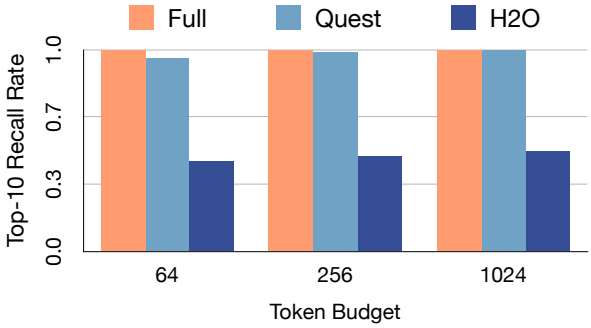

Figure 4. Recall rate of tokens with Top-10 attention scores. Results are profiled with LongChat-7b-v1.5-32k model in passkey retrieval test of 10K context length. Recall rate is the ratio of tokens selected by different attention methods to tokens selected by the full attention in each round of decoding. The average rate is shown in the figure, with various token budgets assigned.

We then present the design of Quest and discuss its benefits.

3.1. Long-context Inference Is Costly

LLM inference contains two stages, namely, the prefill stage and the decode stage. In the prefill stage, all the input tokens are transformed into embeddings and generate the Key (K), Query(Q), and Value(V) vectors. Both the Key and the Value vectors are saved in the KV cache for future use. The rest of the prefill stage includes self-attention and feed-forward network (FFN) layers, which produce the first response token.

In the decode stage, the model will take the last generated token to calculate its K, Q, V. The model uses Q to multiply with every K of previous tokens to generate the *attention weights*. The attention weights will then get normalized using softmax, where each value a_i represents the attention score between ith token and the current token. The self-attention layer will output $\sum a_i \cdot V_i$ and send to the FFN.

For one request, the prefill stage only happens once, while a decoding process is needed for every token in the response. Therefore, the decode stage dominates the inference time. For example, for 16k token prompts and 512 token responses, over 86% of the time is spent on decode stages. Therefore, the decode stage performance is crucial for overall latency.

Moreover, a long-context scenario significantly slows down the decode stage. In every decode stage, the K and V of existing tokens must be loaded to perform self-attention, which can easily reach 16GB for the 32k context of Llama-7b[†]. This memory load operation can take 53% of the time in a decode stage. Therefore, optimizing self-attention becomes a must for efficient long-context inference.

[†]KV cache size = 2 (both K and V) * Num of Layer * Sequence length * Num of Heads * Head Dimensions * Size of FP16 $= 2 * 32 * 32 * 32 * 128 * 2 = 16$GB

3.2. Self-Attention Operation Features High Sparsity

Luckily, previous research has highlighted the inherent sparsity in self-attention (Zhang et al., 2023b; Ge et al., 2024). Due to this property of self-attention, a small portion of tokens in the KV cache, called critical tokens, can accumulate sufficient attention scores, capturing the most important inter-token relationships. For example, as shown in Fig. 3, apart from the first two layers, less than 10% of the tokens are needed to achieve similar accuracy, which makes the attention on the rest of the tokens unnecessary. Therefore, if we can estimate the criticality of the tokens, we can only compute self-attention on critical KV cache tokens to greatly reduce the memory movement and thus improve efficiency.

3.3. Critical Tokens Depend on the Query

However, the criticality of the tokens is dynamic and highly dependent on the query vector Q. Assuming the prompt is "A is B. C is D. A is", we demonstrate the attention map of a certain head in the 16th layer of Llama-2-7b in Fig. 2. Since the output answer here should be "B", the token "B" is critical to the current query "is". Thus, it has a high attention score. However, before the final token "is", "B" is not critical for any previous query and has very low attention scores. In other words, the criticality of tokens is tightly related to the query token.

We quantify this effect by profiling the average recall rate of tokens with Top-10 attention scores along the text generations. The original attention with full KV cache can maintain 100% recall rate. However, KV cache eviction algorithm like H2O (Zhang et al., 2023b) which prunes tokens based on history information, suffers from low recall